51

BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

Livelock, Thread Starvation & Priority Inversion: Sessiz Katiller

Livelock, Thread Starvation & Priority Inversion: The Silent Killers

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
                    LIVELOCK VS STARVATION VS DEADLOCK

1. DEADLOCK (ÖLÜ):
   İki thread tamamen UYUR (BLOCKED). CPU kullanımı = %0. İlerleme = 0.

2. LIVELOCK (AŞIRI HAREKETLİ AMA SONUÇSUZ):
   İki thread sürekli birbirine yol verir. CPU kullanımı = %100! İlerleme = 0.
   Thread 1: "Önce sen geç!" ──► Geri çekilir.
   Thread 2: "Hayır sen geç!" ──► Geri çekilir. (Sonsuz kibarlık döngüsü!)

3. STARVATION (AÇLIK):
   Açgözlü (Greedy) yüksek öncelikli thread'ler CPU'yu bırakmaz.
   Düşük öncelikli işçi thread günlerce kuyrukta bekler, asla sıra gelmez!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Livelock, dar bir koridorda karşı karşıya gelen iki aşırı kibar insanın hikayesidir. Biri sağa kaçar, diğeri de aynı anda onun sağına kaçar. Sonra ikisi birden sola kaçar. Sürekli ter döker, adım atarlar (CPU %100 yanar) ama ikisi de koridordan geçemez! Starvation (Açlık) ise açık büfe kuyruğunda öndeki VIP müşterilerin sürekli araya tanıdıklarını alması yüzünden arkadaki garibanın akşama kadar tek bir tabak yemek alamamasıdır.

🇬🇧 ENGLISH (ELI5)

Livelock is two overly polite pedestrians trying to pass each other in a narrow corridor. Both step to the left simultaneously. Both apologize and step to the right simultaneously. They are sweating profusely and sprinting back and forth (CPU 100%), but making zero forward progress! Starvation is standing at the back of a cafeteria line where VIPs continuously cut in ahead; the poor worker at the rear starves forever without receiving a meal.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Üç Sessiz Katilin Mekaniği: 1) Livelock: Genellikle deadlock'u önlemek için yazılan `tryLock` mantığının yanlış uygulanmasıyla ortaya çıkar. Thread A kilidi alamayınca kendi kilidini bırakır; Thread B de aynı anda bırakır; sonra aynı anda tekrar almaya çalışırlar. Çözüm: Çarpışma anında rastgele gecikme eklemektir (Jitter / Exponential Backoff). 2) Thread Starvation: Adil olmayan (non-fair) kilitler veya CPU'yu hiç bırakmayan sonsuz döngülü thread'ler yüzünden düşük öncelikli görevlerin kaynak bulamamasıdır. Çözüm: `ReentrantLock(true)` adil kilit modu veya adil iş kuyruklarıdır. 3) Priority Inversion (Öncelik Tersinmesi): Düşük öncelikli bir thread kilit tutarken, orta öncelikli bir thread CPU'yu kapar ve düşük önceliklinin kilidi bırakmasını engeller; böylece yüksek öncelikli thread dolaylı olarak orta öncelikli tarafından rehin alınır! Çözüm: Öncelik Devralma (Priority Inheritance) protokolleridir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Silent Killers Analyzed: 1) Livelock: Frequently induced by naive retry loops paired with `tryLock()`. Both threads detect contention, release held locks simultaneously, wait an identical fixed interval, and attempt re-acquisition, looping indefinitely at 100% CPU. Remedy: Introduce randomized truncated exponential backoff (jitter). 2) Starvation: High-priority threads or unfair synchronization mechanisms (`synchronized` or non-fair `ReentrantLock`) continuously bypass threads parked in wait queues. Remedy: Deploy fair synchronization (`new ReentrantLock(true)`) or work-stealing schedulers. 3) Priority Inversion: Occurs when a low-priority thread holds a shared lock, an un-synchronized medium-priority thread preempts it on the CPU, preventing the low-priority thread from releasing the lock needed by a stalled high-priority thread. Remedy: Priority Inheritance Protocols.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Rastgele Gecikme (Jitter) ile Livelock'u Engelleme
import java.util.concurrent.ThreadLocalRandom;
import java.util.concurrent.locks.ReentrantLock;

public class LivelockResolutionDemo {
    private final ReentrantLock lockA = new ReentrantLock();
    private final ReentrantLock lockB = new ReentrantLock();

    public boolean executeWithJitter(String workerName) throws InterruptedException {
        while (true) {
            if (lockA.tryLock()) {
                try {
                    if (lockB.tryLock()) {
                        try {
                            System.out.println(workerName + " iki kilidi de aldı, kritik iş
yapılıyor!");
                            return true;
                        } finally {
                            lockB.unlock();
                        }
                    }
                } finally {
                    lockA.unlock();
                }
            }

            // ⚠ LIVELOCK ÇÖZÜMÜ: Eşzamanlı çakışmada rastgele bekle (Jitter Backoff)
            int jitterMs = ThreadLocalRandom.current().nextInt(10, 50);
            System.out.printf("%s kilit alamadı, %d ms rastgele uyuyor...%n", workerName, jitterMs);
            Thread.sleep(jitterMs);
        }
    }

    void main() throws Exception {
        Thread.ofVirtual().start(() -> {
            try { executeWithJitter("Worker-1"); } catch (Exception _) {}
        });
        Thread.ofVirtual().start(() -> {
            try { executeWithJitter("Worker-2"); } catch (Exception _) {}
        });
        Thread.sleep(200);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

ReentrantLock'ı her zaman adil modda (fair=true) başlatmak performansı dramatik şekilde düşürebilir (throughput 10x-20x azalabilir)! Açlık sorunu gerçekten kanıtlanmadığı sürece varsayılan haksız (non-fair) modu koruyun.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Enabling fair mode (`new ReentrantLock(true)`) imposes severe throughput degradation (often 10x-20x slower) due to context switches and mandatory FIFO queuing. Reserve fair locks strictly for workloads where starvation is an empirically proven threat.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Livelock %100 CPU harcayıp ilerleyememektir; rastgele jitter ile çözülür. Açlık (starvation) ise bazı thread'lere hiç sıra gelmemesidir.

🎯 Key Takeaways for the Interview

• Livelock spins at 100% CPU without progress; resolve via randomized jitter. Starvation occurs when unfair queues starve low-priority threads.